
软件测试

Outline

- Bug Story
- 什么是软件测试?
- 软件测试的分类
- 软件测试技术
- 软件测试活动
- 软件测试的度量

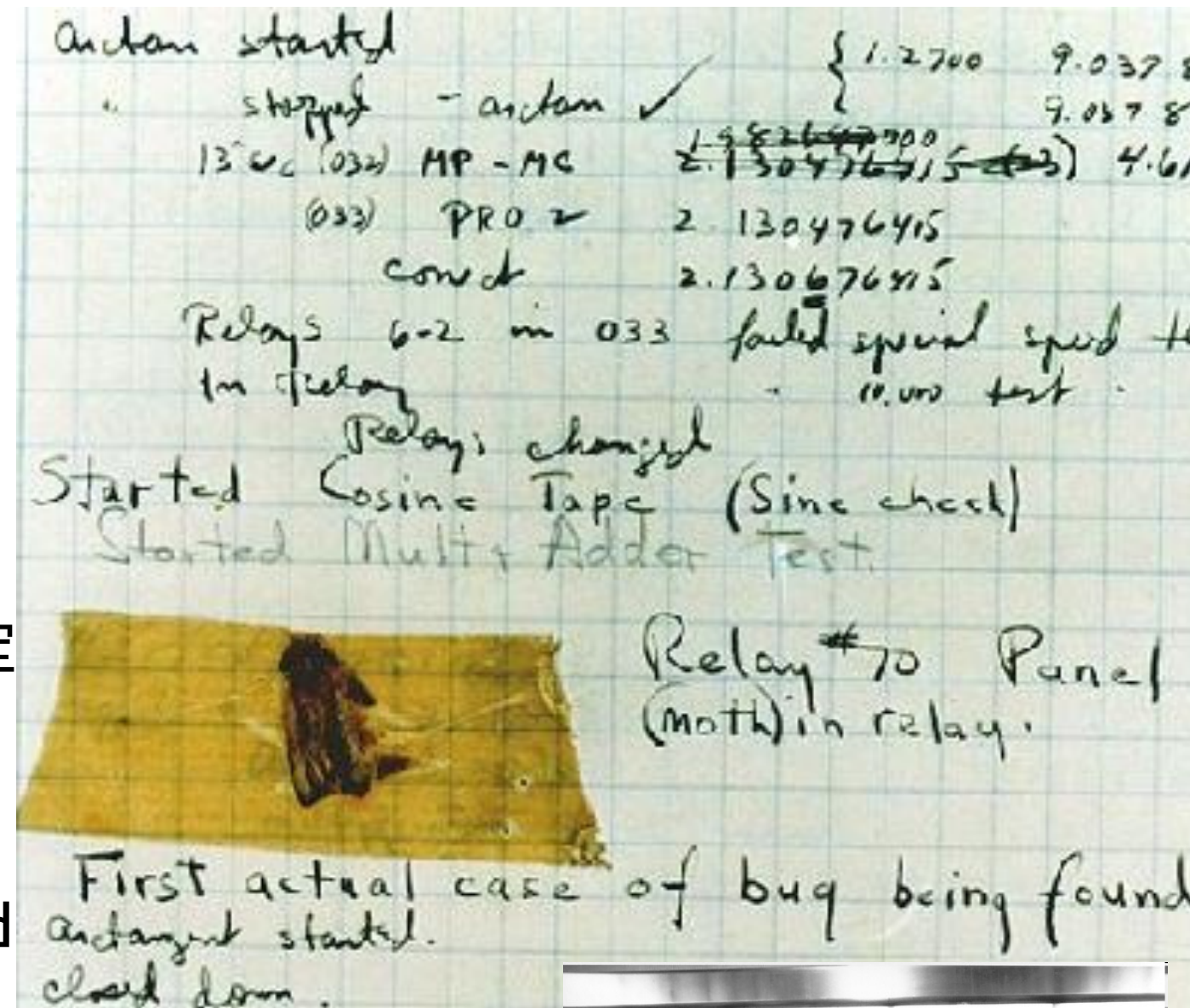
First Bug

Grace Murray Hopper(1906-1992)
计算机科学家 美国海军将军

编译器的发明者

COBOL语言的开发负责人

ACM Grace Murray Hopper Award



Millennium bug (Y2K)

- In the 1970s and on, programmers saved precious memory and disk space by storing years in a two-digit format
 - They (we) never dreamed that their programs would still be running in 2000



Radiation machine kills four: 1985 to 1987

- Faulty software in a Therac-25 radiation-treatment machine resulted in several cancer patients receiving lethal overdoses of radiation. Four patients died
- An investigation by independent scientists found that accidents occurred even after Atomic Energy of Canada Limited (AECL) thought it had fixed particular bugs
- They wrote in their report, "The basic mistakes here involved poor software-engineering practices and building a machine that relies on the software for safe operation."



AT&T long distance service fails: 1990

- Switching errors in AT&T's call-handling computers caused the company's long-distance network to go down for nine hours, the worst of several telephone outages in the history of the system
- The meltdown affected thousands of services and was eventually traced to a single faulty line of code



at&t

Patriot missile defense 1991

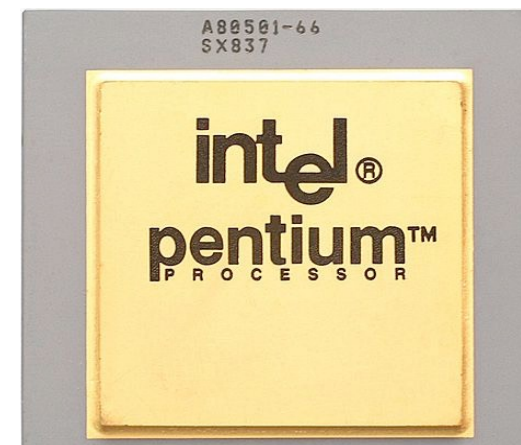
- The Patriot missile defense system failed to defend against several missiles
- A small timing error in the system's clock accumulated so that after 14 hours, the tracking system was no longer accurate
 - In the attack on Dhahran, KSA, the system had been in operation for over 100 hours
- The system was mobile, so the developers did not expect it to remain in one place for 14 hours



Pentium FDIV bug

$$(4195835/3145727) * 3145727 - 4195835 \neq 0$$

- Test engineers found bug
 - Management decided it was not severe
- A user reported it 10/30/1994, Intel attempted to play down the bug. Offered to replace chips if you could prove you were affected
- Ultimately, they apologized; replaced all chips, cost \$450 million
- Interestingly, 1.13 MHz Pentium III caused software freezes



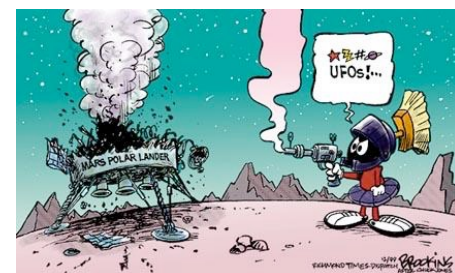
Disney's "Lion King Animated Storybook" 1994

- Disney released its first multimedia CD-ROM for children
- A major ad campaign touting it as “the game to buy”
- After Christmas Day, many angry parents were on the phone to Disney with crying children in the background
- The problem was the game worked on only a few platforms and nothing was said about this on the wrapper as Disney hadn't realized the problem



NASA's Mars polar lander

- 1999, the lander disappeared during landing attempt. A switch on the landing gear's foot set a bit intended to cut off fuel to the landing thruster at touch down; But the switch was tripped already when the legs snapped open for landing
 - Team I tested the leg fold-down procedure, never checked the bit as it was not in the area
 - Team II tested the rest of the landing procedure; reset the computer, hence clearing the bit
 - Both parts worked perfectly separately, but failed when they needed to work together - and that was never tested! They never noticed that the touch-down bit remained set after fold-down



什么是对的软件？

- Are you building it “right”?
- Are you building the “right” thing?

Are you building it “right”?

- Software Correctness
 - A program P is considered with respect to a specification S , if and only if:
 - For each valid input, the output of P is in accordance with the specification S
- Verification

Are you building the “right” thing?

- 常我们会将软件与终端用户或者其他利益相关者的需求进行比较。
- Validation

Verification and Validation

- Verification: is the system correct with respect to some specification?
- Validation: did we build the right system?
- V&V differences don't matter
- V&V generally refers to any activity that attempts to ensure that the software will function as required

If software not correct...

- Definitions
 - Fault / Defect – The cause of a failure, e.g., a missing or incorrect piece of code
 - Error – An intermediate unstable state
 - Failure – The manifested inability of the program to perform the function required, i.e., a system malfunction evidenced by incorrect output, abnormal termination or unmet time and space constraints.
- Relationship
 - A fault may remain undetected long time, until some event activates it.
 - If an when the error propagates to the output, it eventually causes the failure.
- Fault Error Failure
 - This chain can recursively iterate: a fault in turn can be caused by the failure of some other interacting system.

Common V&V Activities

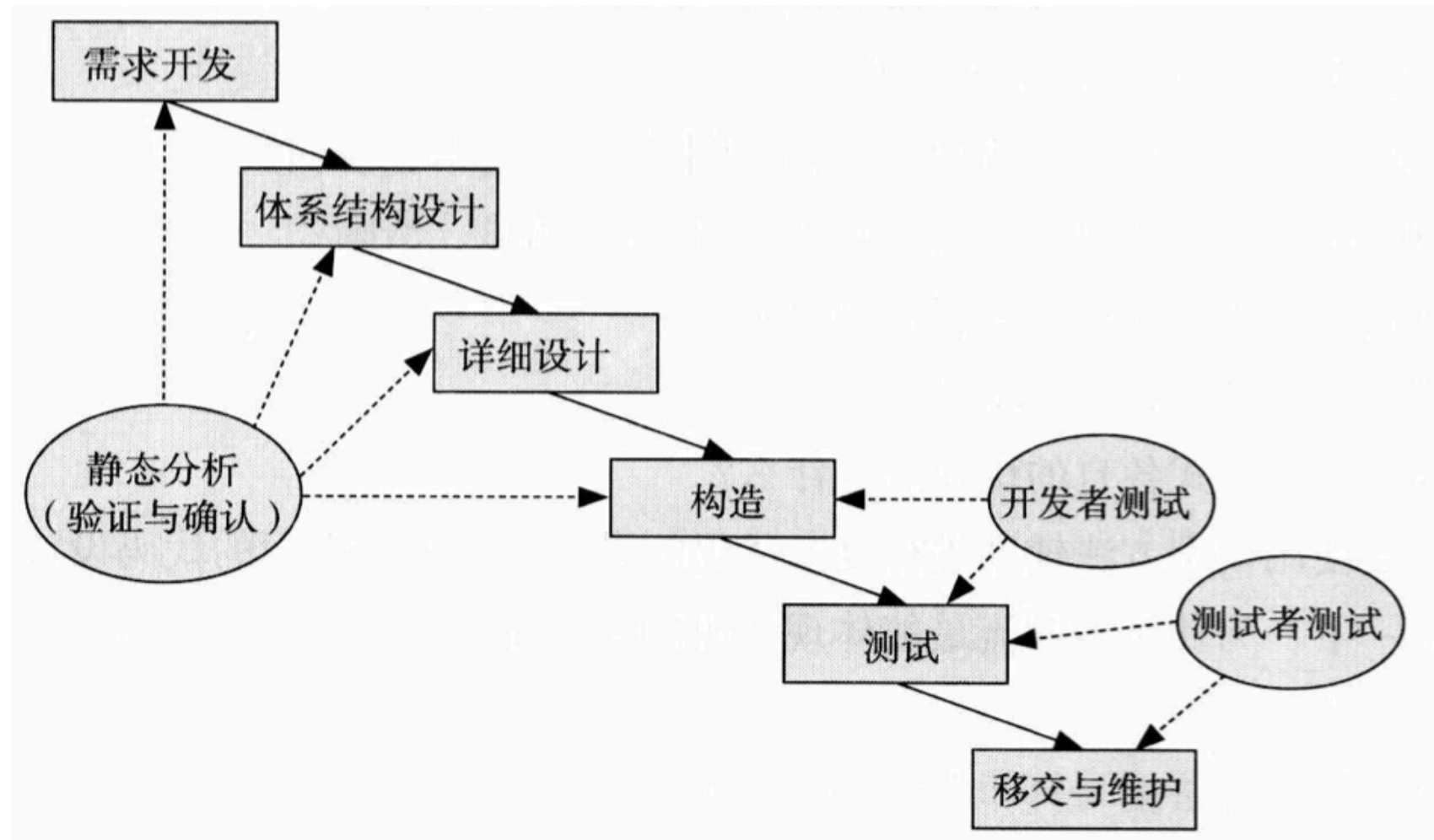
- Static Analysis Techniques –
 - Based solely on the (manual or automated) examination of project documentation of software models and code, and of other related information about requirements and design
 - Generally yields valid results, but may be weak in precision
- Dynamic Test Techniques –
 - Exercise the software in order to expose possible failures
 - Behavioral and performance properties of the program are also observed
 - Yields more precise results, but only holding for the examined executions

Static Techniques

- Can be applied at the requirements phase; at the design phase; and during the implementation phase
- Traditional Static Techniques – heavily manual, error-prone, time consuming
 - Software inspection – The step-by-step analysis of the documents (deliverables) produced, against a compiled checklist of common and historical defects
 - Software reviews – The process by which different aspects of the work product are presented to project personnel (managers, users, customer etc) and other interested stakeholders for comment or approval
 - Code reading – The desktop analysis of the produced code for discovering typing errors that do not violate style or syntax
 - Algorithm analysis and tracing – The process in which the complexity of algorithms employed and the worst case, average-case and probabilistic analysis evaluations can be derived
- Static Analysis Techniques relying on use of Formal Methods

Dynamic Techniques

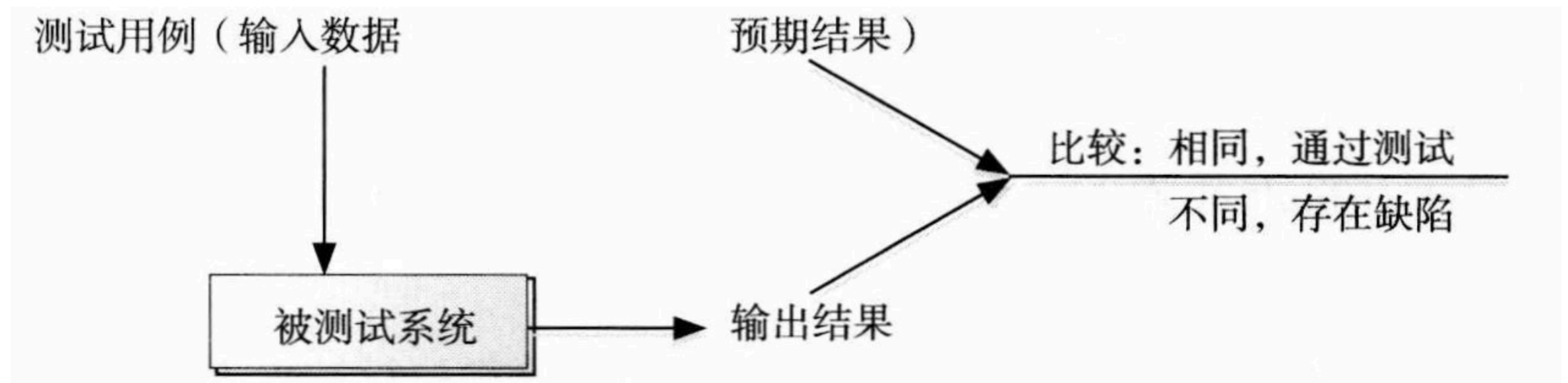
- Dynamic Techniques –
 - Testing – Based on the execution of the code on valued inputs (definition of the parameters and environmental conditions that characterize a system state must be included when necessary)
 - Profiling – A program profile records the number of times some entities of interest occur during a set of controlled executions



软件开发中的验证与确认活动

软件测试的目的

- 向开发者和用户展示软件满足了需求，表明软件产品是一个合格的产品；
 - 有效性测试
- 找出软件中的缺陷和不足。
 - 缺陷测试

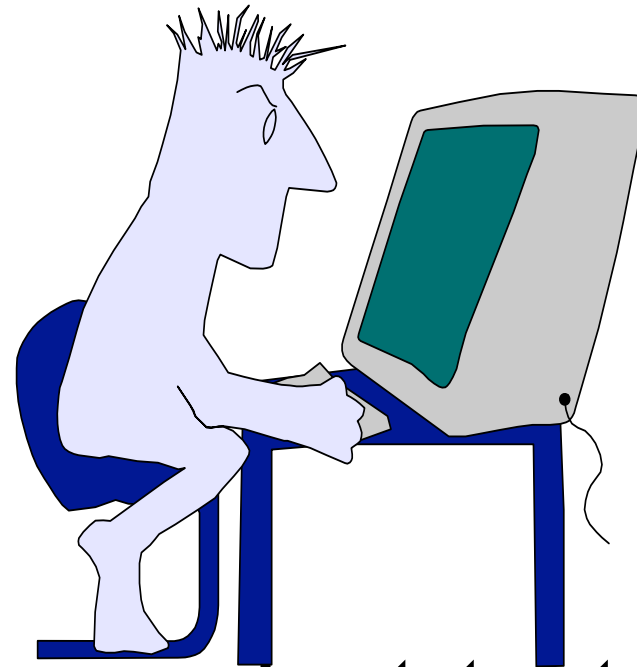


测试用例



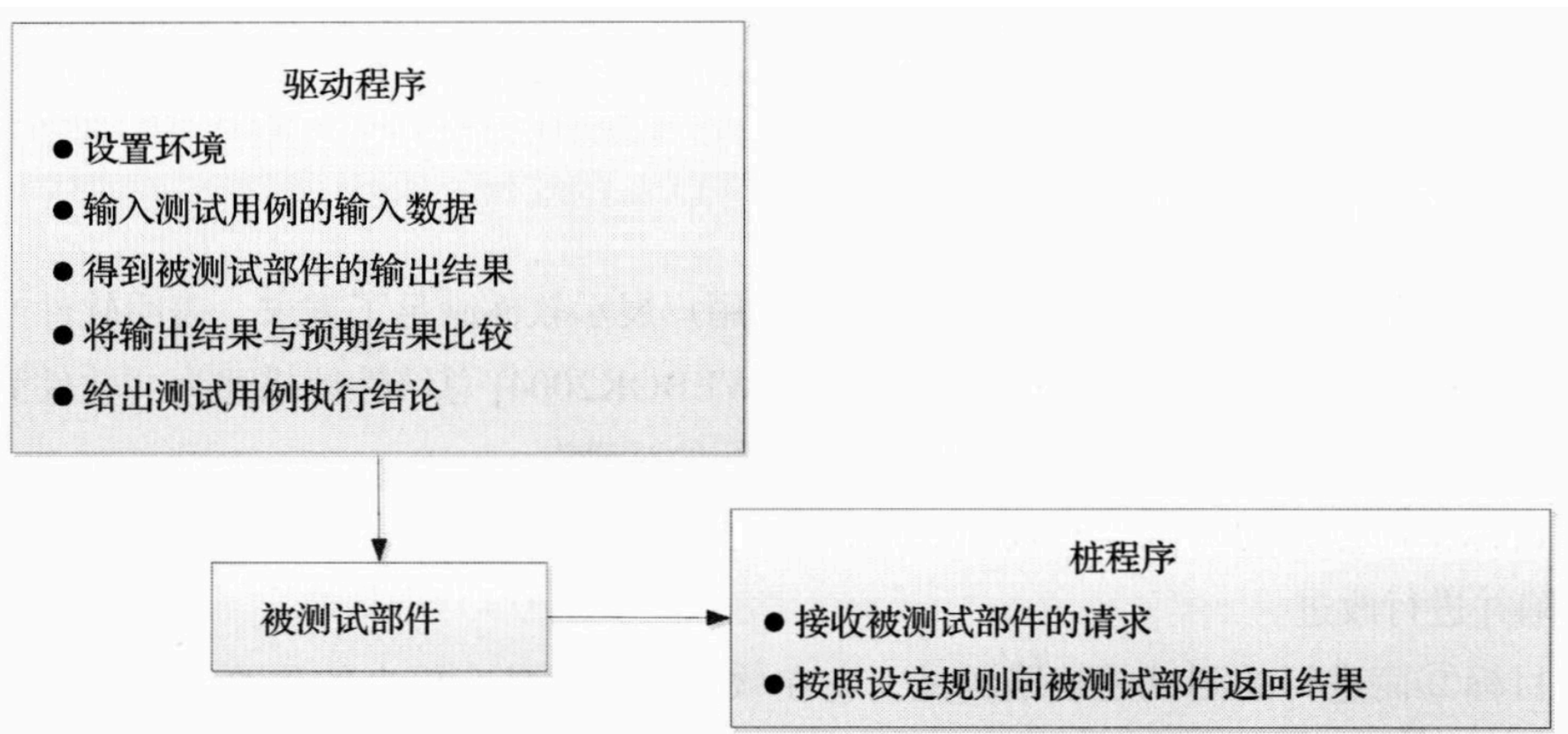
developer

Understands the system but,
will test "gently" and, is driven
by "delivery"



independent tester

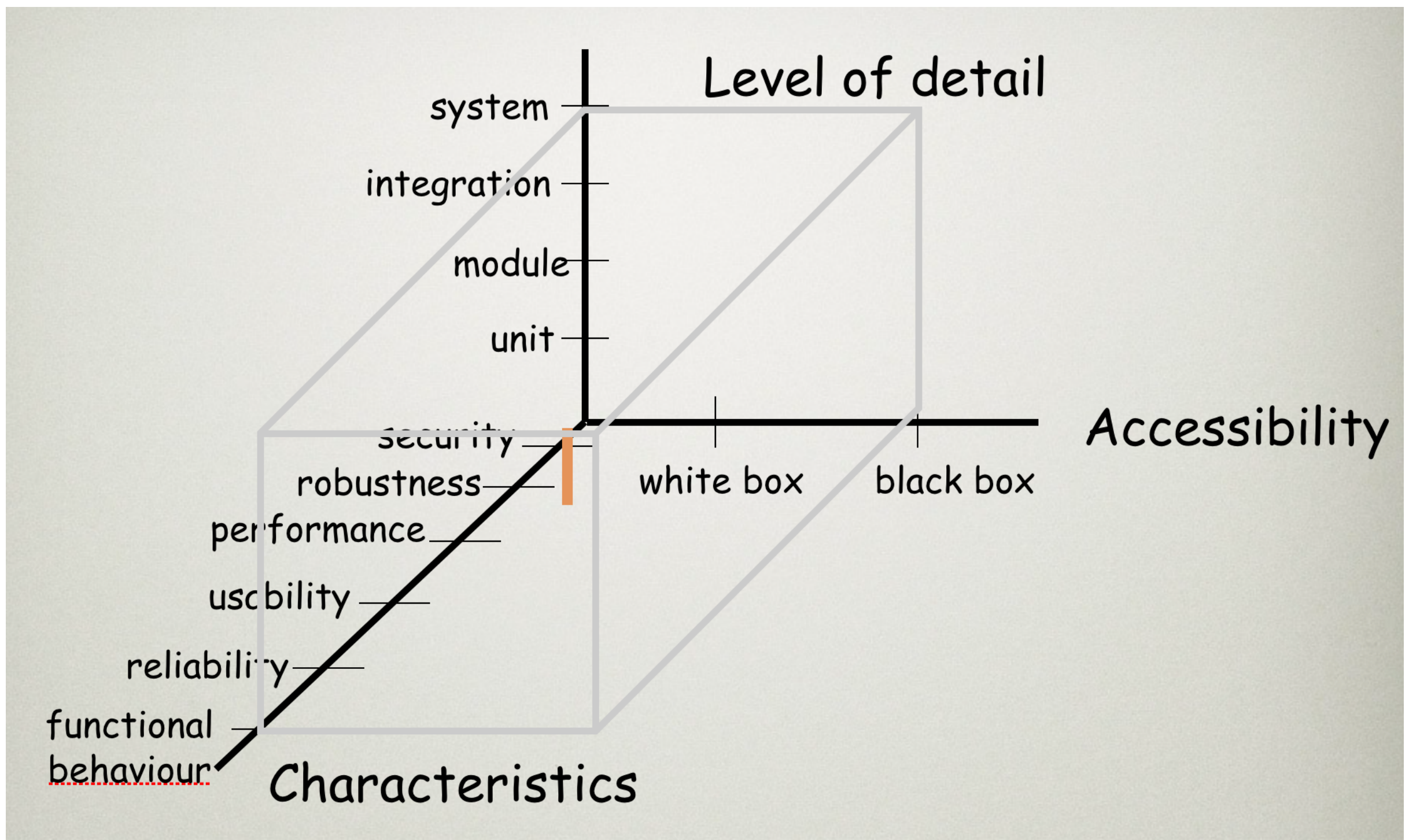
Must learn about the
system, but, will attempt
to break it and, is driven
by quality



桩与驱动

桩程序驱动程序

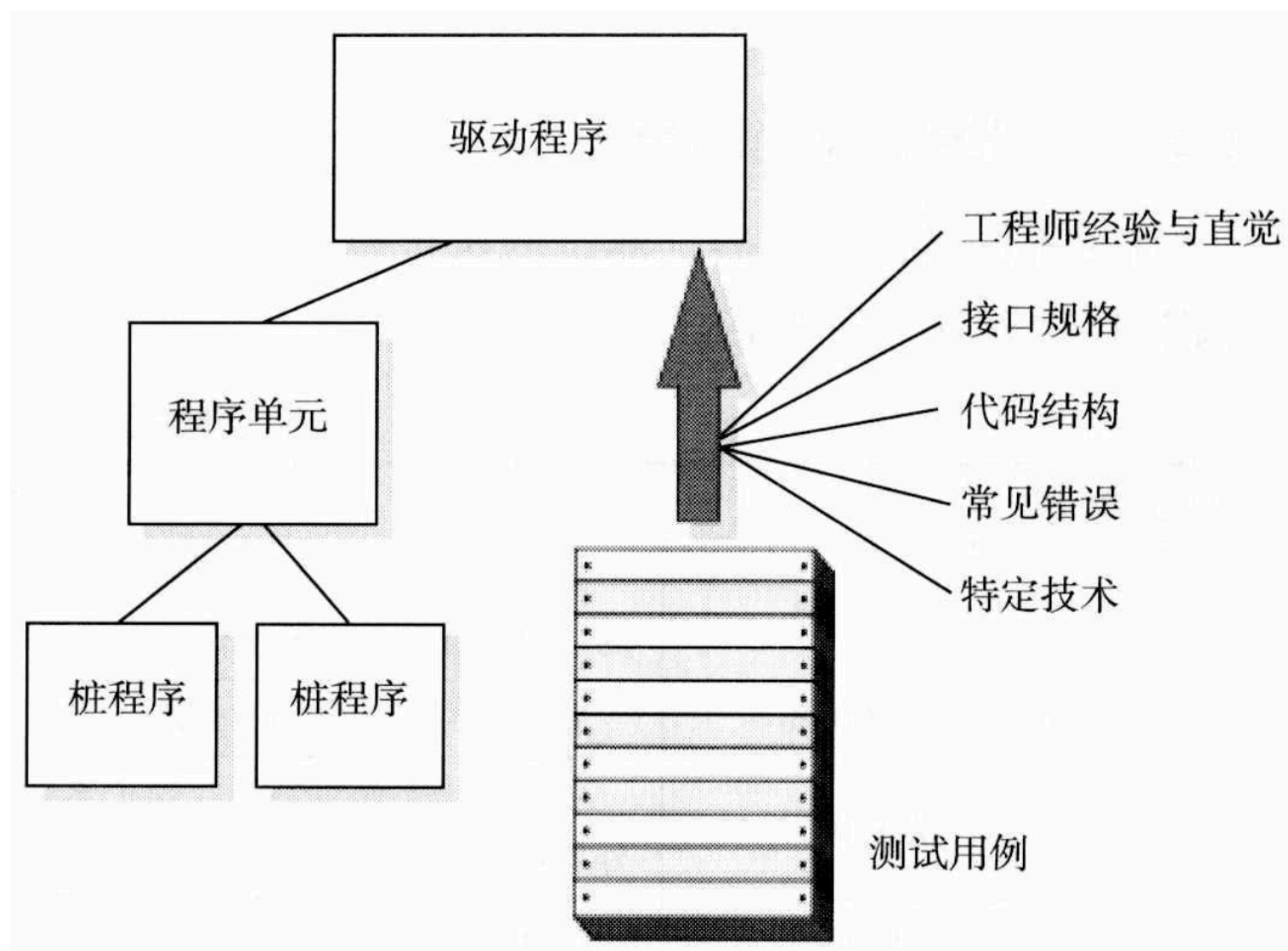
- 桩程序是被测试部件的交互环境，它扮演被测试部件需要调用的其他系统部件。桩程序对其他系统部件的扮演仅限于规格相同，内部代码要简单地多，通常是直接返回固定数据或者按照固定规则返回数据。
- 驱动程序负责创建被测试部件的执行环境，并驱动和监控被测试部件执行测试用例的过程，判定测试用例的执行结果。



TESTING Space

表 19-1 测试层次

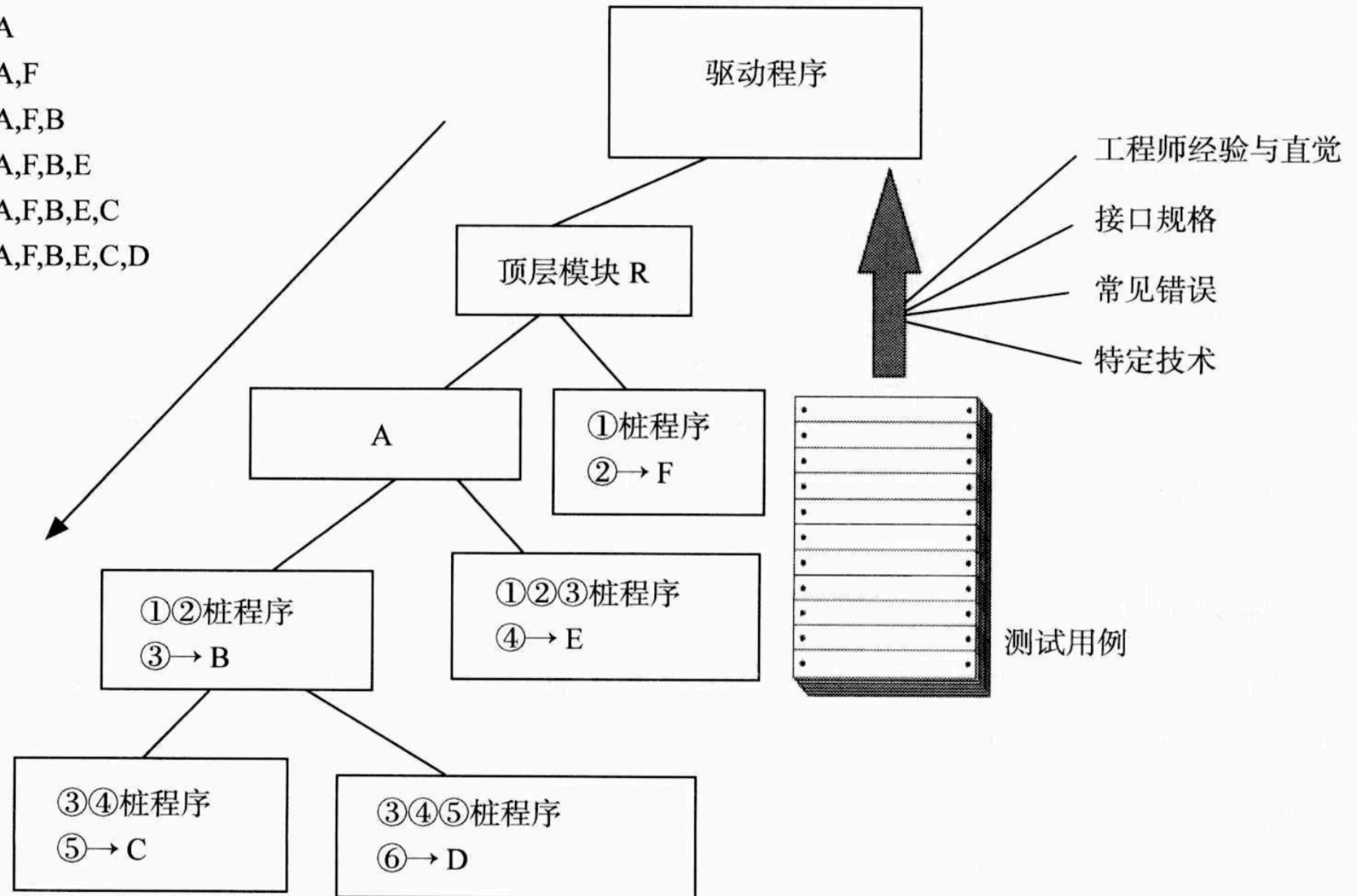
	类别	描述
测试对象	单元测试	验证独立软件片段的功能，软件片段可以是单个的子程序、或者是由紧密联系的单元组成的较大的组件。
	集成测试	验证软件组件之间的交互。
	系统测试	关注整个系统的行为，评价系统功能需求和非功能性需求，也评价系统与外界环境（例如其它应用、硬件设备等）的交互。
测试目标	功能测试	确认观察到的被测试软件的行为是否遵从软件需求规格说明。
	验收测试	按照客户的需求检查系统。这个测试活动可能需要开发人员的参与，也可能不需要他们的参与。
	安装测试	在目标环境中通过安装来验证软件。
	α 与 β 测试	在软件发布前，让小规模、有代表性的潜在用户试用，可以在开发机构中进行（ α 测试），也可在用户处进行（ β 测试）。通常， α 与 β 测试不需要控制。
	性能测试	特别针对性能需求验证软件。
	易用性测试	评价终端用户学习和使用软件（包括用户文档）的难易程度、软件功能在支持用户任务的有效程度、从用户的错误中恢复的能力。
	可靠性测试	验证和评价系统可靠性的测试。
	安全测试	验证系统内的安全机制保护系统不受非法入侵的能力。
	恢复测试	验证软件在“灾难”后的重新启动能力。
	压力测试	以设计的最大负载运行软件，并以超过最大负载运行软件，验证软件的负载能力。
	配置测试	分析软件在规格说明的不同配置下的行为。
	回归测试	在变更系统后进行，重新执行已经测试过的测试用例集，以确保变更没有造成未预期的副作用。每个测试级别的测试（不论何种测试对象、功能还是非功能需求）都可以进行回归测试。
	其他暂未列举的测试类型.....	



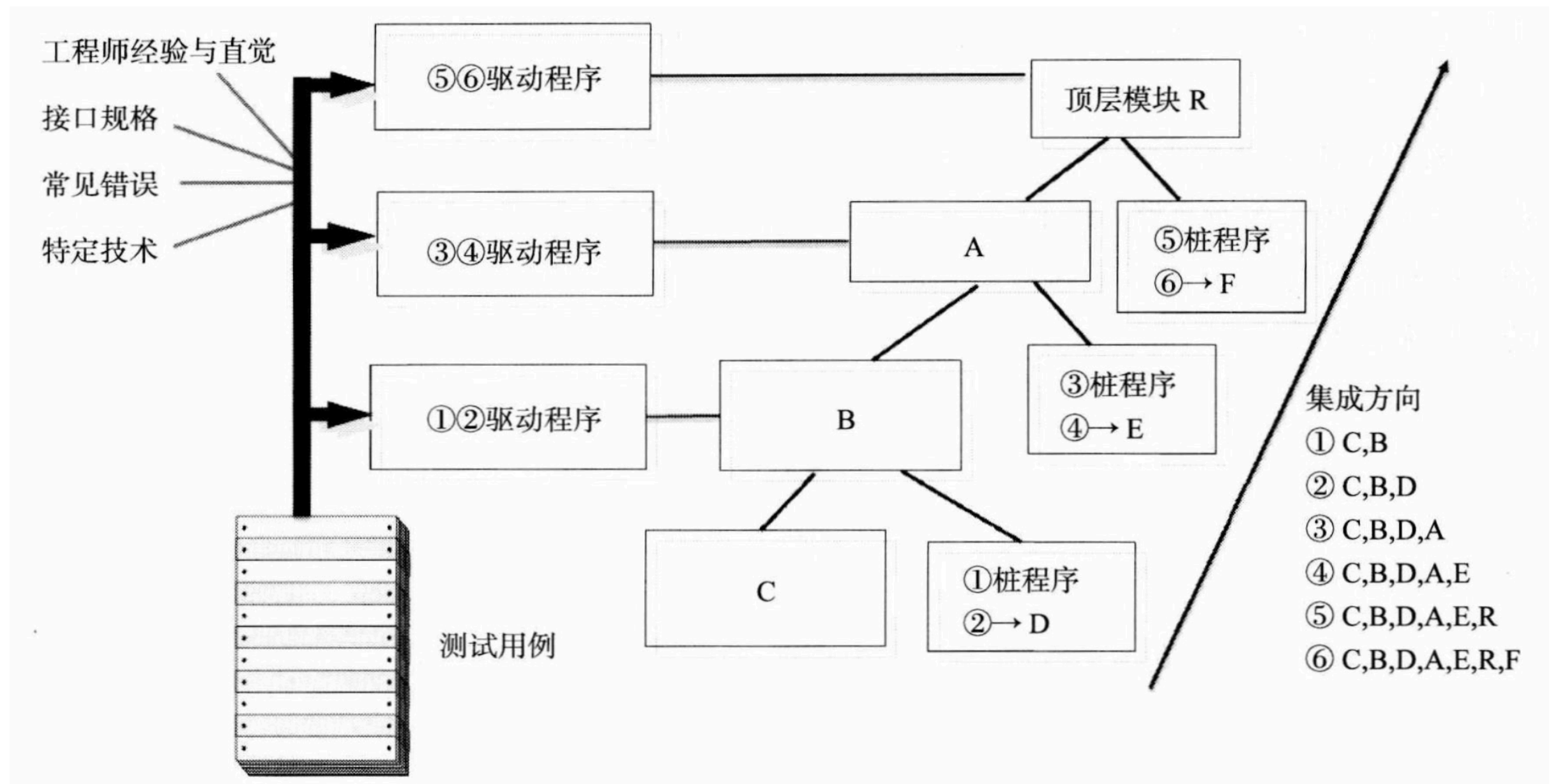
单元测试

集成方向

- ① R,A
- ② R,A,F
- ③ R,A,F,B
- ④ R,A,F,B,E
- ⑤ R,A,F,B,E,C
- ⑥ R,A,F,B,E,C,D



自顶向下的集成测试



自底向上的集成测试

系统测试

- 单元测试、集成测试更加关注技术上的正确性，重在发现设计缺陷和代码缺陷。系统测试则不同，它更关注不符合需求的缺陷和需求自身的内在缺陷。
- 根据测试目标的不同，有很多不同类型的系统测试：功能测试、非功能性测试、验收测试、安装测试等等。但是发生在软件测试阶段，完全由软件测试人员控制和执行的主要是功能测试和非功能性测试。
- 系统测试关注整个系统的行为，所以不依赖于桩程序和驱动程序。但是，使用一些测试工具可以让系统测试过程更加自动化。
- 系统测试的功能测试计划以需求规格说明文档或用例文档为基础，主要使用随机测试和基于规格的测试技术设计功能测试用例。在测试非功能性需求时需要使用针对非功能需求的特定测试技术进行测试计划和测试用例设计。

测试技术

- 测试用例的选择
- 随机测试
- 基于规格的技术 - 黑盒测试方法
- 基于代码的技术 - 白盒测试方法
- 特定测试技术

测试用例的选择

- 测试的目标是发现尽可能多缺陷， 并不绝对要求所有。
- 因为测试是有代价的， 不仅要耗费桩程序、驱动人力等成本， 更重的是随着测试用例数量增多会直线上升。
- 所以， 软件测试人员需要仔细地选择用例在代价尽可能小的情况下发现足够多的缺陷 [Zhu1997]。
- 测试技术就是帮助软件人员设计和选择用例的。

随机测试

- 随机测试（随机测试Ad hoc Testing）是一种基于软件工程师直觉和经验的技术，也许是实践中使用最为广泛的测试技术 [SWEBOK2004] 。
- 随机测试 根据软件工程师的技能、直觉和对类似程序的经验[Myers1979]，从所有可能的输入值中选择输入子集，建立测试用例。

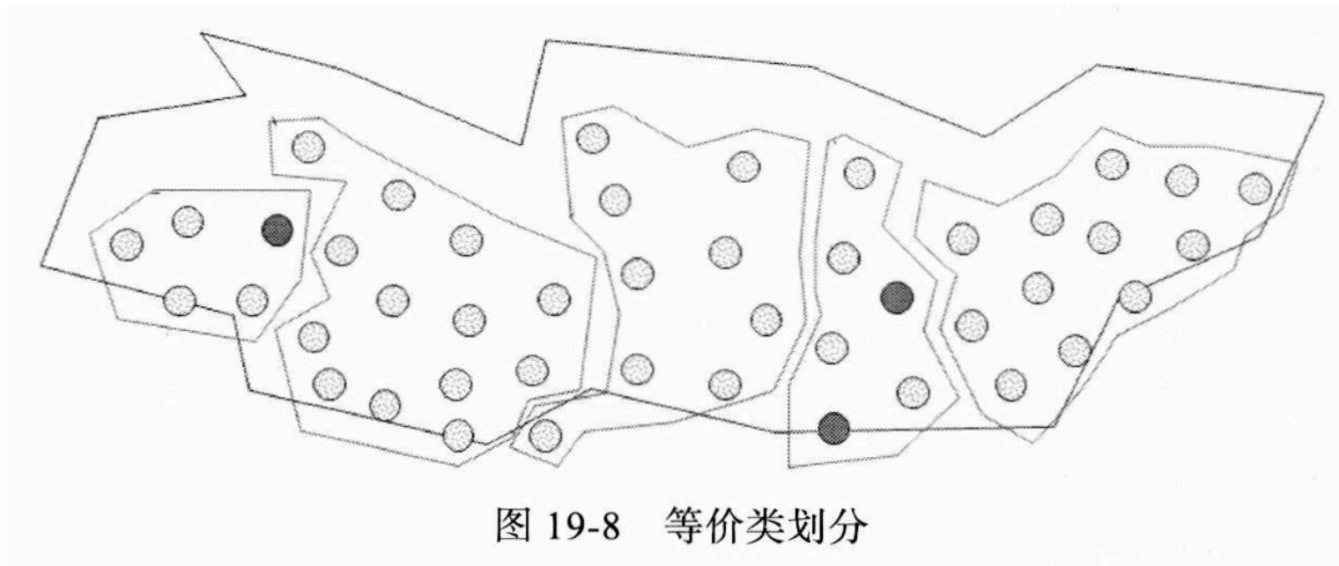
表 19-2 函数 int add(int x, int y) 的测试用例

ID	输 入		预 期 输 出
1	x=100	y=20	120
2	x= 2147483640	y=100	超出最大值异常

黑盒测试方法

- 等价类划分

有效等价类：是指对于程序的规格说明来合理、意义输入数据构成集合。利用有效等价类可检验程序是否实现了规格说明中所定的功能和性能。



Sale类

getChange(double payment)方法

所示的规格说明，可以将其输入数据划分为三类：

- 1有效数据；
- 2无效数据， $\text{payment} \leq 0$
- 3无效数据， $\text{payment} < \text{total}$

表 19-3 Sales 类 getChange(double payment) 方法的等价类划分测试用例

ID	输入		预期输出
1	payment=100	Total=50	50
2	payment=-100	total=20	输入数据无效
3	payment=50	total=100	输入数据无效

黑盒测试方法

- 边界值分析

表 19-4 Sales 类 getChange(double payment) 方法的边界值分析测试用例

ID	输入		预期输出
1	payment=50	total=50	0
2	payment=50	total=49	1
3	payment=50	total=51	输入数据无效
4	payment=1	total=1	0
5	payment=0	total=0	0
6	payment=-1	total=10	输入数据无效

- 1) $\text{prePoint} < 1000 \&\& \text{postPoint} \geq 1000 \rightarrow \text{GiftLevel} = 1;$
- 2) $\text{prePoint} < 2000 \&\& \text{postPoint} \geq 2000 \rightarrow \text{GiftLevel} = 2;$
- 3) $\text{prePoint} < 5000 \&\& \text{postPoint} \geq 5000 \rightarrow \text{GiftLevel} = 3。$

表 19-5 礼品赠送事件的测试用例

ID	输 入		预 期 输 出
1	prePoint=500	postPoint=1500	GiftLevel=1
2	prePoint=500	postPoint=2500	GiftLevel=2
3	prePoint=500	postPoint=5500	GiftLevel=3

黑盒测试方法 - 决策表

黑盒测试方法 - 状态转换

- 使用状态转换测试技术时，通常要先为对象建立状态图，描述测试对象的状态集合、输入集合和输入导致的状态转换集合。
- 以状态图基础，可建立测试对象的转换表。状态转换表每一行都应该被设计为测试用例。

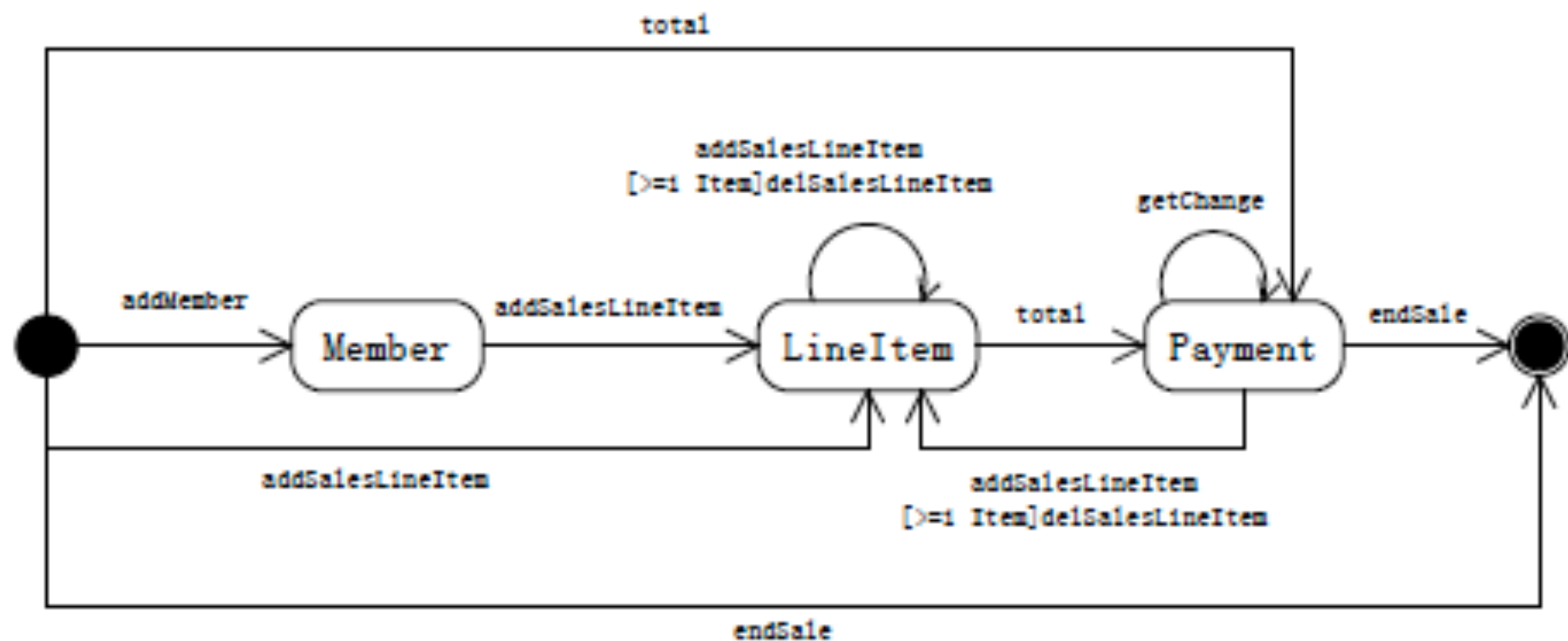


表 18-4 类 Sales 的测试用例线索

输入		预期输出状态
方法	当前状态	
<i>addMember</i>	<i>Start</i>	<i>Member</i>
	<i>Member</i>	非法
	<i>LineItem</i>	非法
	<i>Payment</i>	非法
	<i>End</i>	非法
<i>addSalesLineItem</i>	<i>Start</i>	<i>LineItem</i>
	<i>Member</i>	<i>LineItem</i>
	<i>LineItem</i>	<i>LineItem</i>
	<i>Payment</i>	<i>LineItem</i>
	<i>End</i>	非法
<i>[>=1 Item] delSalesLineItem</i>		
<i>total</i>		
<i>getChange</i>		
<i>endSale</i>		

表 18-5 类 Sales 的测试用例

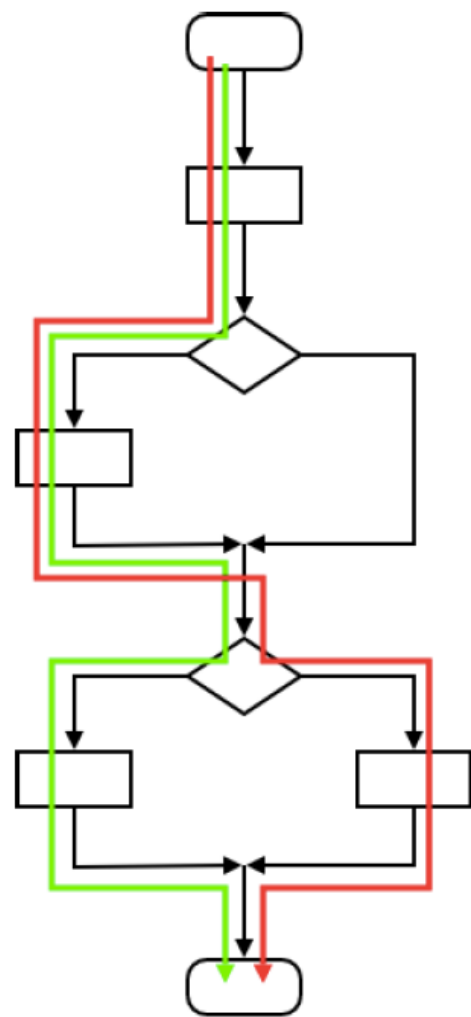
ID	輸入		預期輸出
	前置語句	方法	
1	<i>s=new Sales();</i>	<i>s.addMember(2);</i>	<i>No Exception</i>
2	<i>s=new Sales();</i> <i>s.addMember(1);</i>		<i>MemberLable</i> <i>Invalid Time</i>
3	<i>s=new Sales();</i> <i>s.addSalesLineItem(1);</i>		
4	<i>s=new Sales();</i> <i>s.addSalesLineItem(1);</i> <i>s.total();</i>		
5	<i>s=new Sales();</i> <i>s.addSalesLineItem(1);</i> <i>s.total();</i> <i>s.getChange(100);</i> <i>s.endSale();</i>		<i>Sales dose not Exists</i>

白盒测试方法

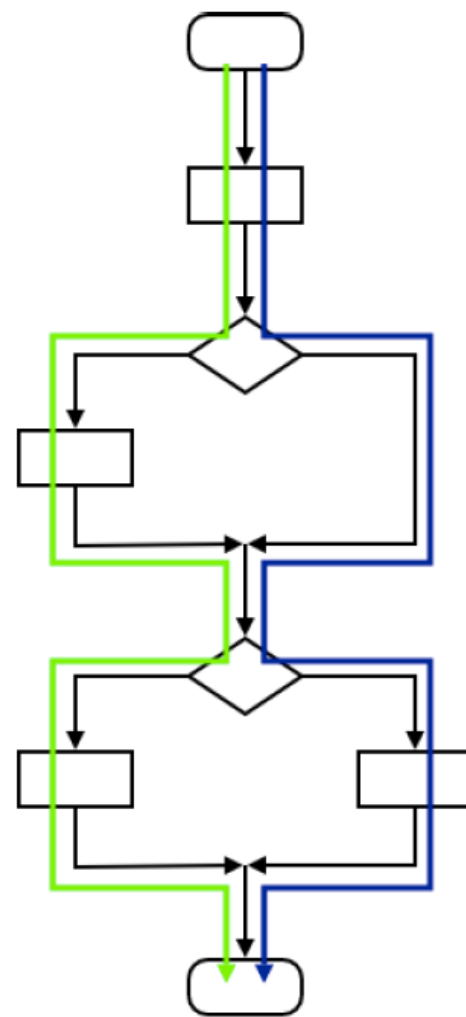
- 语句覆盖

语句覆盖设计测试用例的标准是确保被测试对象的每一行程序代码都至少执行一次。

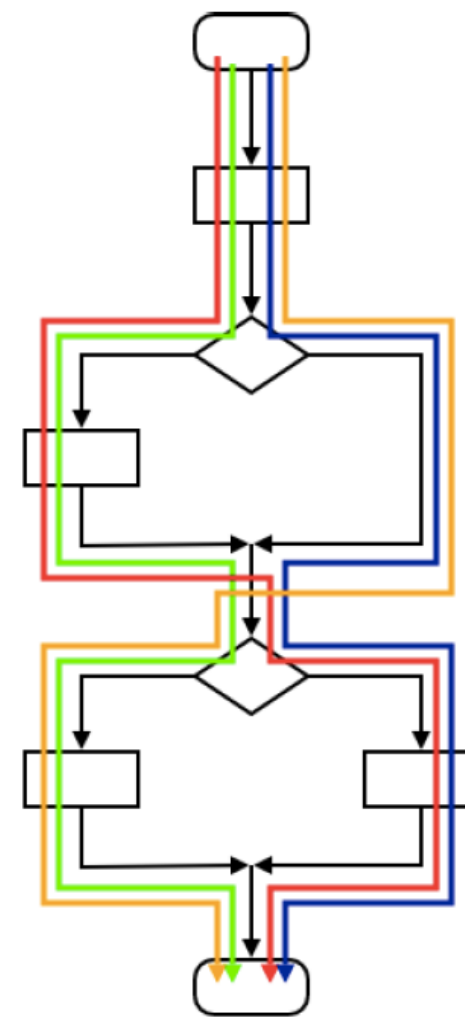
```
public Class Customer {  
    int bonus;//积分数额  
    .....  
    //预计算消费行为后的积分数额  
    int getBonus ( boolean cashPayment , int consumption , bo  
        //已有的 bonus 需要调整 getBonus, 不能直接使用  
        int preBonus=this.getBonus();  
        if (cashPayment) {  
            preBonus+=consumption;  
        }  
        if (vip) {  
            preBonus*=1.5;  
        } else {  
            preBonus*=1.2;  
        }  
        return preBonus;  
    }  
}
```



Statement Coverage
(aka line coverage)

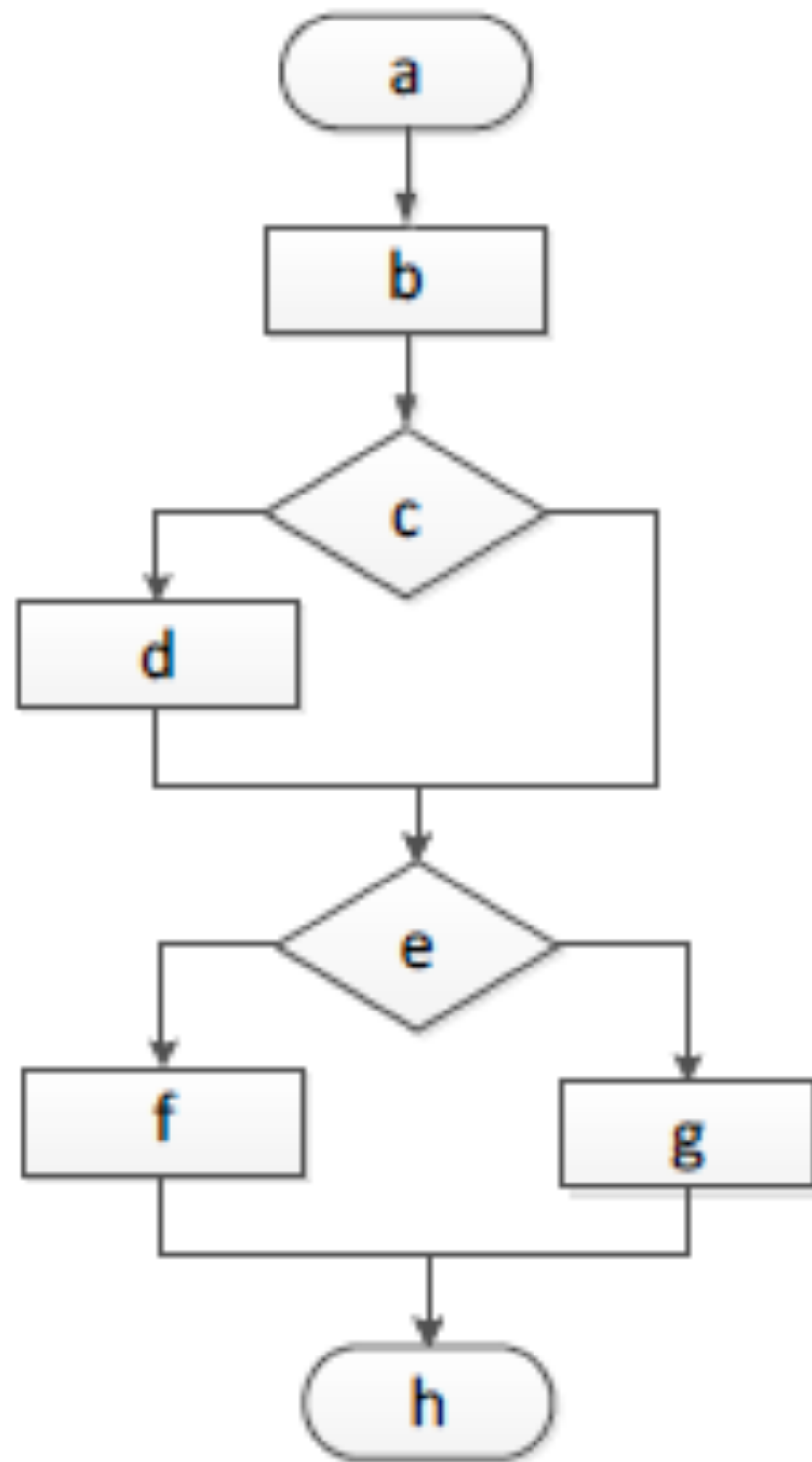


Branch Coverage
(aka condition coverage)



Path Coverage

白盒测试



为了清晰地解释对语句的覆盖，图19-10为各条语句都进行了编号。可以发现如果按照
下列两条路径执行，就能够覆盖所有的语句：
a,b,c,d,e,f,h 和
a,b,c,d,e,g,h

表 19-6 Customer.getBonus() 的语句覆盖测试用例

ID	输入				预期输出
1	<i>getBonus ()=100</i>	<i>cashPayment=true</i>	<i>consumption=100</i>	<i>vip=true</i>	300
2	<i>getBonus ()=100</i>	<i>cashPayment=true</i>	<i>consumption=100</i>	<i>vip=false</i>	240

白盒测试方法

- 条件覆盖

条件覆盖设计测试用例的标准是确保程序中每个判断的每个结果都至少满足一次

表 19-7 Customer.getBonus() 的条件覆盖测试用例

ID	输入				预期输出
1	<code>getBonus ()=100</code>	<code>cashPayment=true</code>	<code>consumption=100</code>	<code>vip=true</code>	300
2	<code>getBonus ()=100</code>	<code>cashPayment=false</code>	<code>consumption=100</code>	<code>vip=false</code>	120

白盒测试方法

- 路径覆盖

路径覆盖测试用例的标准是确保程序中每条独立执行路径都至少执行一次

表 19-8 Customer.getBonus() 的路径覆盖测试用例

ID	输入				预期输出
1	<code>getBonus ()=100</code>	<code>cashPayment=true</code>	<code>consumption=100</code>	<code>vip=true</code>	300
2	<code>getBonus ()=100</code>	<code>cashPayment=false</code>	<code>consumption=100</code>	<code>vip=false</code>	120
	<code>getBonus ()=100</code>	<code>cashPayment=true</code>	<code>consumption=100</code>	<code>vip=false</code>	240
	<code>getBonus ()=100</code>	<code>cashPayment=false</code>	<code>consumption=100</code>	<code>vip=true</code>	150

特点测试技术

- (1) 面向对象的测试；
- (2) 图形用户接口GUI测试；
- (3) 基于Web的测试；
- (4) 基于组件的测试
- (5) 并发程序的测试；
- (6) 协议遵从性测试；
- (7) 实时系统测试；
- (8) 极端要求安全性的系统的测试。

面向对象测试技术

- 在本书中使用了一些面向对象的测试技术：
 - (1) 7.5使用用例和场景设计系统测试用例；
 - (2) 11.5基于协作设计类之间的集成测试用例；
 - (3) 18.5基于状态图设计类的单元测试用例。

测试活动

- 测试计划
- 测试设计
- 测试执行
- 测试评价

表 19-9 测试用例日志

测试用例 ID	种 类	条 件	期 望 结 果	测 试 结 果	测试对象 ID

测试用例日志

表 19-10 缺陷报告

缺陷 ID	发现日期	测试脚本	测试用例	期望结果	实际结果	状态	严重性	优先级	缺陷类型	备注

缺陷报告

测试层次的测试报告

1.引言

1.1 文档标识

文档的唯一标识，可以包含日期、单位、作者、状态、版本等信息。

1.2 范围

文档的内容及组织方式。

1.3 参考资料

2.详细情况

2.1 测试结果概述

总结对具体测试项目的评价。

标明测试项目（对象），注明它们的层级，描述测试活动执行的环境和环境带来的影响。

2.2 详细测试结果

汇总测试的结果。

- 标明所有已解决的缺陷，并总结它们的分析辨别（Resolution）。
- 标明所有未解决的缺陷。对于推迟解决的缺陷，解释后续的处理办法。
- 总结主要的测试活动和事件。总结收集的相关度量数据。
- 报告测试对象与它们规格之间的任何差异。描述每个差异的原因。

如果有相应的测试计划，就依据测试计划中的综合性标准进行测试过程的综合性评价。

2.3 决策理由

描述影响总结决定的相关事宜。

2.4 总结与建议

为每一个测试对象：

- 描述整体评价，包括它的局限性。评价以测试结果和测试项目所处层次的通过/失败标准为基础。评价还可能包括对失败风险的估计。
- 建议如果要投入生产使用应该具备的状态和所处的环境。

3.其他

3.1 术语表

3.2 文档修改历史

图 19-11 [EEE829-2008]建议的具体测试层次的测试报告

总测试报告

1.引言

1.1 文档标识

文档的唯一标识，可以包含日期、单位、作者、状态、版本等信息。

1.2 范围

文档的内容及组织方式。

1.3 参考资料

2.详细情况

2.1 总测试结果概述

总结每一个测试层次的评价信息：

- 测试活动总结：描述所有测试活动的执行汇总情况。
- 测试任务的结果总结：描述所有的测试任务情况。
- 缺陷和辨析（Resolution）总结：分类总结测试中发现的缺陷。已解决的缺陷和未解决的缺陷应该分开总结。
- 评估产品质量：对产品进行总体评价。
- 总结收集的度量数据。

2.2 决策理由

描述对软件作出“通过/不通过/有条件通过”决定的原因。

2.3 总结与建议

- 描述对产品的总体性评价。评价可能会包括对失败风险的估计。
- 建议如果产品要投入生产使用应该具备的状态和所处的环境。
- 提供关于产品验收的总结和建议。总结包括确定产品准备就绪需要的精力（effort）汇总。建议要标明判定产品已就绪的指示特征。
- 描述学习到的任何经验。这部分可能会包含过程改进的信息。
- 如果有缺陷被推迟处理，要解释后续处理过程。

3.其他

3.1 术语表

3.2 文档修改历史

图 19-12 [IEEE829-2008]建议的总测试报告

测试度量

- 缺陷数据
- 测试覆盖率
 - 需求覆盖率
 - 模块覆盖率
 - 代码覆盖率

缺陷度量

- 根据引入缺陷的阶段
 - 系统需求缺陷、设计缺陷和编码缺陷
- 根据缺陷的影响力
 - 严重、一般和无影响

表 19-11 测试覆盖率与程序规模

规模（代码行）	测试用例数量	测试覆盖率
1	1	100%
10	2	100%
100	5	95%
1000	15	75%
10 000	250	50%
100 000	4000	35%
1 000 000	50 000	25%
10 000 000	350 000	15%

测试覆盖率与程序规模

覆盖率

- (1) 需求覆盖率 = 被测试的需求数量 / 需求总数;
- (2) 模块覆盖率 = 被测试的模块数量 / 模块总数;
- (3) 代码覆盖率 = 被测试的代码行 / 代码行数总数。